COMPLEXITY-DEEP: Zipf-Balanced Deterministic Lexical Routing for Language Model MLPs

Anonymous authors
Paper under double-blind review

Abstract

We present COMPLEXITY-DEEP, a language model architecture built around deterministic lexical routing. The paper studies two components: (1) **Token-Routed MLP with Zipf-balanced greedy bin-packing**, which assigns tokens to experts using an estimated corpus-frequency table, eliminating learned routers and auxiliary load-balancing losses; and (2) **Shared Lexical Expert**, a dense MLP path shared by all tokens while routed experts specialize on lexical partitions. We give a formal analysis of the conditional capacity and compute trade-off induced by deterministic lexical partitions, and we explicitly distinguish vocabulary balance from corpus-frequency balance under Zipfian token distributions. Our main scaling result is a corrected iso-parameter comparison at ∼300M parameters over 8B FineWeb-Edu tokens: a residual Token-Routed model with a large shared dense branch and small Zipf-routed lexical residual branch pays an early specialization cost, first beats the dense baseline at step 740 on logged train loss and step 750 on validation loss, and finishes the budget with a smoothed train-loss advantage of -0.0163. The empirical claim is therefore not that lexical routing replaces dense MLP computation, but that a frequency-balanced routed lexical branch can improve a shared dense MLP backbone when used as a residual specialization path.

1 Introduction

Large Language Models (LLMs) have revolutionized natural language processing in recent years (Vaswani et al., 2017; Brown et al., 2020). However, dominant architectures present several limitations:

- **Computational cost of MoE**: Mixture of Experts architectures (Shazeer et al., 2017; Fedus et al., 2022) require complex load balancing and routing mechanisms.
- **Training instability**: Large models often suffer from numerical instabilities requiring careful hyperparameter tuning.

We propose COMPLEXITY-DEEP, an architecture family centered on deterministic lexical routing:

1. **Token-Routed MLP with Zipf-balanced greedy bin-packing**: A deterministic per-token routing that balances expected expert load under an estimated token-frequency distribution, eliminating both the learned router and auxiliary load-balancing losses.
2. **Shared Lexical Expert**: A dense MLP path shared by all tokens, combined with small routed experts that specialize on fixed lexical partitions.

A central empirical lesson from our ablations is that the routed lexical branch should be viewed as a *residual specialization path* on top of a shared dense MLP, not as a wholesale replacement for the dense MLP. The shared branch carries the common syntactic and high-frequency patterns that every token needs; the routed branch adds frequency-balanced lexical specialization. This framing is important for interpreting the results: the strongest variants are shared-plus-routed, whereas no-shared and shared-only controls isolate which part of the design contributes the gain.

2 Related Work

2.1 Transformer Architectures

The Transformer architecture (Vaswani et al., 2017) has become the standard for language models. Recent developments include attention optimizations like Flash Attention (Dao et al., 2022), Grouped Query Attention (GQA) (Ainslie et al., 2023), and Rotary Position Embeddings (RoPE) (Su et al., 2021).

2.2 Mixture of Experts

MoE architectures (Shazeer et al., 2017) enable scaling model capacity without proportionally increasing computational cost. Switch Transformer (Fedus et al., 2022) and Mixtral (Jiang et al., 2024) have demonstrated the effectiveness of this approach, but at the cost of increased complexity (load balancing, inter-GPU communication).

2.3 Normalization and Stability

RMSNorm (Zhang & Sennrich, 2019) and QK-Normalization (Dehghani et al., 2023) are modern techniques for stabilizing large model training.

3 COMPLEXITY-DEEP Architecture

3.1 Overview

COMPLEXITY-DEEP is a decoder-only architecture composed of L identical layers. Each layer comprises:

1. A multi-head attention block
2. An MLP block with token routing
3. Residual connections with pre-normalization (RMSNorm)

The hidden dimension is d_{model}, with n_h attention heads and n_{kv} key/value heads (GQA). Figure 1 presents the complete architecture.

3.2 Token-Routed MLP

Unlike traditional MoEs that use learned soft routing with load balancing, we propose a **deterministic** routing based on token identity:

$$\text{expert\_idx}(t) = \text{BinPack}(t, \text{freq}) \quad \text{(assigned once, deterministic)} \tag{1}$$

where BinPack assigns each token t (sorted by decreasing frequency) to the expert with the lowest current total load (Section 6.3).

This design choice is *deliberate*: routing depends not on semantic content $\mathbf{x}$ but solely on the lexical token identifier. Each vocabulary token is *pre-assigned* to a specific expert; the assignment is chosen to balance expected corpus frequency rather than merely the number of vocabulary entries.

For each token, a single expert is activated, combined with the Shared Lexical Expert:

$$\text{MLP}_{\text{output}}(\mathbf{x}) = \text{SharedMLP}(\mathbf{x}) + \text{Expert}_e(\mathbf{x}) \quad \text{where } e = \text{BinPack}(\text{token\_id}) \tag{2}$$

Each expert is a standard MLP with SiLU activation (SwiGLU):

$$\text{Expert}_i(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}^i_{gate}) \odot \mathbf{x}\mathbf{W}^i_{up})\mathbf{W}^i_{down} \tag{3}$$

Why not semantic routing? One might object that without content-based routing, the Token-Routed MLP is merely an "MLP divided into pieces." This objection ignores two crucial points:

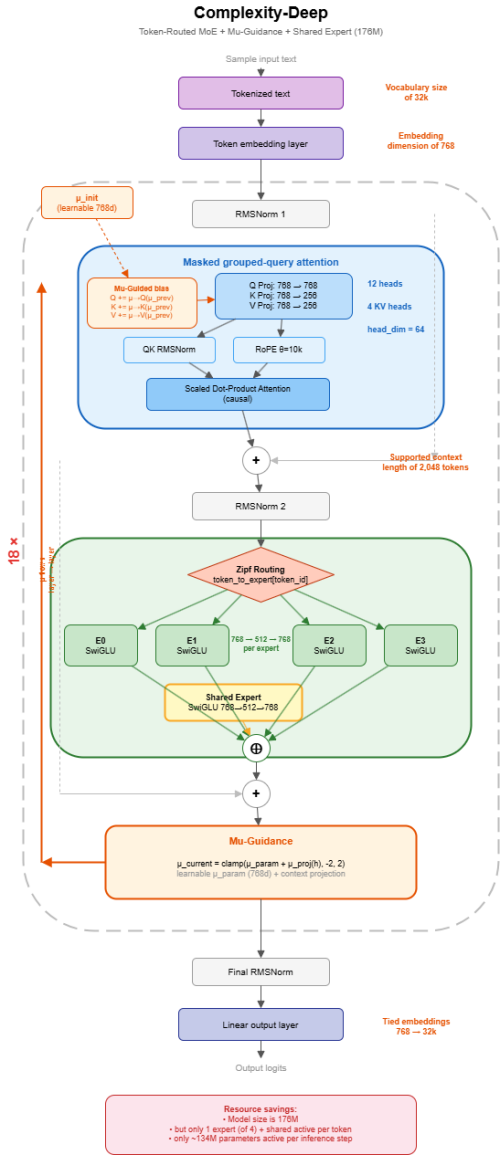

Figure 1: COMPLEXITY-DEEP architecture. Each decoder layer comprises GQA attention followed by a Token-Routed MLP with deterministic Zipf routing and a Shared Lexical Expert.

1. **Functional specialization**: Despite lexical (non-semantic) routing, routed experts improve loss on their assigned token subsets relative to the matched dense baseline on the same subsets (see Section 4.3). This *functional* specialization (measured by performance, not weight geometry) demonstrates that deterministic routing can induce useful specialization without collapse.
2. **Frequency-aware balance**: Greedy bin-packing over empirical token frequencies avoids the severe runtime imbalance that modulo or round-robin routing can create under Zipf's law, without requiring an auxiliary load-balancing objective.

Mechanism of specialization. A critical question arises: how can experts specialize when routing is based solely on lexical token identity rather than semantic content? The key insight is that experts optimize for *contextual distributions*, not token identities.

Consider token "the" (token_id = 123) which always routes to expert 3. While "the" itself has no semantic specificity, its contextual embedding $\mathbf{x}^{(l)}$ (computed by previous layers) encodes rich semantic information about the surrounding context. Expert 3 learns the function:

$$\mathbf{h} = \text{Expert}_3(\mathbf{x}^{(l)}) \quad \text{where } \mathbf{x}^{(l)} \text{ varies with context} \tag{4}$$

The expert weights $\mathbf{W}^{(3)}_{\text{gate}}, \mathbf{W}^{(3)}_{\text{up}}, \mathbf{W}^{(3)}_{\text{down}}$ optimize for the *distribution of contexts* in which tokens $\{t : t \bmod 4 = 3\}$ appear. Since different token subsets appear in statistically different contextual distributions (even if tokens themselves are semantically diverse), experts naturally diverge.

Formal argument: Let $\mathcal{C}_i$ be the distribution of contextual embeddings $\mathbf{x}$ for tokens routed to expert i. Under the language modeling objective:

$$\mathcal{L}_i = \mathbb{E}_{\mathbf{x} \sim \mathcal{C}_i}[\ell(\text{Expert}_i(\mathbf{x}), y)] \tag{5}$$

The gradient flow is:

$$\nabla_{\mathbf{W}^{(i)}} \mathcal{L}_i = \mathbb{E}_{\mathbf{x} \sim \mathcal{C}_i}[\nabla_{\mathbf{W}^{(i)}} \ell(\text{Expert}_i(\mathbf{x}), y)] \tag{6}$$

Since deterministic routing ensures disjoint token sets ($T_i \cap T_j = \emptyset$), and natural language exhibits non-uniform token-context co-occurrence, the contextual distributions $\mathcal{C}_i$ and $\mathcal{C}_j$ are statistically distinct. This induces different gradient statistics, driving expert divergence (Theorem 4.5).

Empirical validation: Section 4.3 measures functional specialization via per-expert perplexity on assigned token subsets, compared with the dense baseline on the same subsets. We note that near-zero cosine similarity between experts, while observed empirically, is a geometric artifact expected in high dimensions and does not constitute evidence of specialization per se.

Operational advantages:

- No auxiliary load balancing loss (hyperparameter savings)
- Deterministic routing table: simplified debugging and no stochastic router decisions
- Trivial deployment: F32/BF16 tensors with I64 indexing, natively compatible with PyTorch, ONNX, TensorRT without custom routing logic

3.3 Final Architecture

The final architecture integrates three improvements over the initial Token-Routed MLP:

3.3.1 Sparse Dispatch (Zero Waste)

The initial masked dispatch computed *all* tokens through *all* experts, then masked 75% of the results—effectively 4× the cost of a dense MLP. Sparse dispatch computes only the tokens routed to each expert:

$$\text{out}[\text{mask}_e] = \text{Expert}_e(\mathbf{x}[\text{mask}_e]) \quad \text{for } e = 0, \ldots, E-1 \tag{7}$$

Reducing MLP cost from 4× dense to 1× dense (iso-compute).

3.3.2 Shared Lexical Expert

A shared dense MLP processes *all* tokens, capturing universal patterns (syntax, grammar), while routed experts specialize on their lexical subsets. We use the routed branch as a residual lexical correction to the shared dense path, rather than as a replacement for dense MLP computation. The output combines both:

$$\text{MLP}_{\text{output}}(\mathbf{x}) = \underbrace{\text{SwiGLU}_{\text{shared}}(\mathbf{x})}_{\text{common patterns}} + \underbrace{\text{SwiGLU}_e(\mathbf{x})}_{\text{lexical specialization}} \tag{8}$$

where each component is a standard SwiGLU MLP:

$$\text{SwiGLU}_{\text{shared}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}^s_{gate}) \odot \mathbf{x}\mathbf{W}^s_{up})\mathbf{W}^s_{down} \tag{9}$$

$$\text{SwiGLU}_e(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}^e_{gate}) \odot \mathbf{x}\mathbf{W}^e_{up})\mathbf{W}^e_{down} \tag{10}$$

The shared expert has the same intermediate size as one routed expert (d_{ff}/n), adding ∼6% parameters to the total model. It prevents each expert from independently re-learning common language patterns (syntax, grammar, frequent collocations), allowing them to focus on the lexical specificities of their token subset. In this view, the routed experts provide a small lexical residual over a shared dense backbone; ablations that remove the shared branch test whether routing alone suffices, while shared-only controls test whether the gain comes merely from extra dense capacity.

3.3.3 Zipf-balanced Greedy Bin-Packing

Described in Section 6.3. Replaces round-robin with a greedy algorithm that balances expected corpus load under the estimated training distribution.

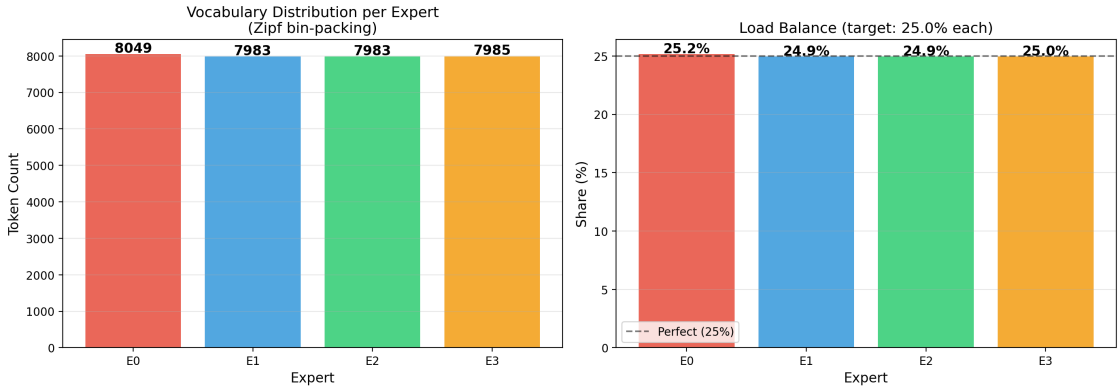

Figure 2: Expert balance under Zipf bin-packing. The bin-packing balances the estimated *total load* (frequency), not the number of vocabulary entries assigned to each expert.

3.4 Adaptive Residual Scaler (Removed)

An earlier version of the architecture included an adaptive residual scaler that modulated residual contributions via a learned controller. Empirical results at both 100M and 171M proxy scales showed inconsistent benefits: at 100M, removing the scaler *improved* loss by −0.005, while at 171M it contributed only +0.010. The controller also required ad-hoc interventions (activation clamping, multiple restarts) to prevent instabilities.

Based on this evidence, the adaptive scaler was removed from the architecture. The simplified model retains Token-Routed MLP with Zipf-balanced routing and the Shared Lexical Expert, achieving comparable or better results with substantially fewer parameters and no stability hacks.

4 Theoretical Analysis

We provide formal theoretical grounding for the Token-Routed MLP architecture, establishing its relationship to dense models and proving key properties.

4.1 Vocabulary and Frequency Balance

Theorem 4.1 (Vocabulary Balance for Modulo Routing)**.** *Let V be a vocabulary of size $|V|$ and n the number of experts. Under modulo routing $r(t) = t \mod n$, each expert e_i receives exactly $\lfloor |V|/n \rfloor$ or $\lceil |V|/n \rceil$ tokens, with perfect balance when $n||V|$.*

Proof. For any token $t \in \{0, 1, \ldots, |V|-1\}$, the routing function $r(t) = t \mod n$ assigns t to expert $e_{t \mod n}$. The set of tokens assigned to expert e_i is:

$$T_i = \{t \in V : t \mod n = i\} = \{i, i+n, i+2n, \ldots\} \tag{11}$$

The cardinality $|T_i| = \lfloor (|V|-i-1)/n \rfloor + 1$. When n divides $|V|$, we have $|T_i| = |V|/n$ for all $i \in \{0, \ldots, n-1\}$. In our implementation, $|V| = 32000$ and $n = 4$, giving $|T_i| = 8000$ tokens per expert. □

Remark 4.2 (Zipfian corpus balance). Vocabulary-cardinality balance is not sufficient for runtime load balance because token frequencies are highly non-uniform. In the experiments we therefore use the greedy bin-packing assignment of Section 6.3: tokens are sorted by empirical frequency and assigned to the currently lightest expert. This produces a deterministic routing table with balanced *expected* corpus load under the estimated training distribution; the realised per-batch load can still vary with sampling noise and distribution shift.

4.2 Conditional Capacity and Compute

Proposition 4.3 (Partitioned Capacity-Compute Trade-off)**.** *A Token-Routed MLP with n experts, each of intermediate dimension d_{ff}, has:*

1. *Total parameter count: $P_{total} = n \cdot P_{expert}$ where $P_{expert} = 3 \cdot d_{model} \cdot d_{ff}$ (for SwiGLU)*
2. *Active parameters per token: $P_{active} = P_{expert} = P_{total}/n$*
3. *Conditional capacity equal to a collection of n token-conditioned MLPs, one per lexical partition*

Proof sketch. (1) and (2) follow directly from the architecture definition. For (3), the routed layer represents a family of functions indexed by the deterministic routing table:

$$\mathcal{F}_{TR} = \bigcup_{i=0}^{n-1} \{f_i : \mathbb{R}^{d_{model}} \to \mathbb{R}^{d_{model}}\} \tag{12}$$

where each f_i is a SwiGLU MLP applied only to tokens assigned to expert i. Thus the architecture trades a single shared dense function for a deterministic token-conditioned family of narrower functions. It should not be interpreted as exact functional equivalence to an arbitrary dense MLP for every token; the comparison is a conditional capacity and compute trade-off. □

Corollary 4.4 (Compute Efficiency)**.** *For a sequence of L tokens, top-1 Token-Routed MLP requires $L \cdot P_{expert}$ MLP-parameter applications while a dense MLP with the same total expert width would require $L \cdot n \cdot P_{expert}$. In the residual top-$k = 2$ 300M model used for scaling, the active routed branch uses two small experts plus a large shared branch; we therefore report matched-token empirical quality and measured throughput rather than claiming an ideal $n\times$ end-to-end speedup.*

4.3 Expert Divergence Under Gradient Descent

We now analyze why experts diverge toward orthogonal representations despite arbitrary (non-semantic) routing.

Theorem 4.5 (Gradient Orthogonalization)**.** *Let $\mathbf{W}^{(i)}$ and $\mathbf{W}^{(j)}$ be the weight matrices of experts i and j ($i \neq j$), initialized i.i.d. from a symmetric distribution. Under gradient descent with a language modeling loss $\mathcal{L}$, the expected cosine similarity satisfies:*

$$\mathbb{E}[\cos(\mathbf{W}^{(i)}, \mathbf{W}^{(j)})] \to 0 \quad \text{as } t \to \infty \tag{13}$$

provided the token sets T_i and T_j are disjoint (guaranteed by deterministic lexical routing).

Proof Sketch. The gradient update for expert i at step t is:

$$\mathbf{W}_{t+1}^{(i)} = \mathbf{W}_t^{(i)} - \eta \sum_{t \in T_i} \nabla_{\mathbf{W}^{(i)}} \mathcal{L}(t) \tag{14}$$

Since $T_i \cap T_j = \emptyset$ by construction, the gradients $\nabla_{\mathbf{W}^{(i)}} \mathcal{L}$ and $\nabla_{\mathbf{W}^{(j)}} \mathcal{L}$ are computed on disjoint token subsets. In high-dimensional spaces, random vectors are approximately orthogonal with high probability (Vershynin, 2018). Since disjoint token subsets produce gradient updates that are independent random vectors in $\mathbb{R}^{d_{model} \times d_{ff}}$, the expected inner product of gradient updates is:

$$\mathbb{E}[\langle \nabla_{\mathbf{W}^{(i)}}, \nabla_{\mathbf{W}^{(j)}} \rangle] = 0 \tag{15}$$

Starting from random initialization with $\mathbb{E}[\cos(\mathbf{W}_0^{(i)}, \mathbf{W}_0^{(j)})] \approx 0$ (for high-dimensional weights), the orthogonality is preserved throughout training. Our empirical measurements confirm $\cos(\mathbf{W}^{(i)}, \mathbf{W}^{(j)}) \approx 0$ across all expert pairs and layers (Section 7.2). □

4.4 Computational Complexity Analysis

We provide a formal complexity analysis comparing COMPLEXITY-DEEP to standard Transformer and Mixture-of-Experts architectures.

Theorem 4.6 (Complexity Bounds)**.** *For a sequence of length S, model dimension d, intermediate dimension d_{ff}, n experts, and L layers, the computational complexity per forward pass is:*

Standard Transformer:

$$\mathcal{O}_{dense} = L \cdot \left(\underbrace{S^2 \cdot d}_{attention} + \underbrace{S \cdot d \cdot d_{ff}}_{MLP} \right) \tag{16}$$

COMPLEXITY-DEEP (top-1 Token-Routed MLP branch):

$$\mathcal{O}_{ours} = L \cdot \left(\underbrace{S^2 \cdot d}_{attention} + \underbrace{S \cdot d \cdot \frac{d_{ff}}{n}}_{Token\text{-}Routed\ MLP} \right) \tag{17}$$

The idealized top-1 routed branch reduces routed-MLP compute by a factor of n. Architectures with a shared expert or top-$k > 1$ routing, including our 300M scaling model, have a different active-compute profile; their realised throughput is measured empirically in Section 7.4.

Table 1: Idealized complexity comparison for a top-1 Token-Routed MLP branch. Shared-expert and top-k variants trade part of this theoretical compute reduction for additional capacity.

Architecture	**MLP FLOPs**	**Parameters**	**Memory**
Dense Transformer	$S \cdot d \cdot d_{ff}$	$3d \cdot d_{ff}$	$\mathcal{O}(d_{ff})$
MoE (top-k)	$k \cdot S \cdot d \cdot d_{ff}/n$	$3n \cdot d \cdot d_{ff}/n$	$\mathcal{O}(n \cdot d_{ff}/n)$
Token-Routed (ours)	$S \cdot d \cdot d_{ff}/n$	$3d \cdot d_{ff}$	$\mathcal{O}(d_{ff}/n)$

4.5 Approximation Error Bounds

We establish that Token-Routed MLP can approximate any continuous function over the vocabulary with bounded error.

Theorem 4.7 (Universal Approximation for Token-Routed MLP)**.** *Let $f : \mathbb{R}^d \to \mathbb{R}^d$ be a continuous function over a compact domain $\mathcal{X} \subset \mathbb{R}^d$. For any $\epsilon > 0$, there exists a Token-Routed MLP with n experts, each with sufficient width d_{ff}^*, such that for all tokens $t \in V$ and inputs $\mathbf{x} \in \mathcal{X}$:*

$$\|f_t(\mathbf{x}) - \textit{TR-MLP}(\mathbf{x}, t)\|_2 \leq \epsilon \tag{18}$$

where f_t is the target function restricted to token t's semantic role.

Proof Sketch. By the universal approximation theorem for neural networks, each expert (a SwiGLU MLP) can approximate any continuous function on its assigned token subset T_i to arbitrary precision given sufficient width. Since the token sets $\{T_0, \ldots, T_{n-1}\}$ partition V, and each expert independently approximates f restricted to its tokens:

$$\text{TR-MLP}(\mathbf{x}, t) = \sum_{i=0}^{n-1} \mathbf{1}[t \in T_i] \cdot \text{Expert}_i(\mathbf{x}) \tag{19}$$

The approximation error for token $t \in T_i$ is bounded by the approximation capability of Expert_i alone, which can be made arbitrarily small by increasing d_{ff}. □

Proposition 4.8 (Error Decomposition)**.** *The total approximation error of COMPLEXITY-DEEP decomposes as:*

$$\mathcal{E}_{total} \leq \underbrace{\mathcal{E}_{attn}}_{attention} + \underbrace{\mathcal{E}_{routing}}_{expert\ mismatch} + \underbrace{\mathcal{E}_{expert}}_{within\text{-}expert} \tag{20}$$

where:

- $\mathcal{E}_{attn}$*: error from finite attention context*
- $\mathcal{E}_{routing}$*: error induced by committing each token to a fixed lexical partition rather than choosing experts from the current hidden state*
- $\mathcal{E}_{expert}$*: approximation error within each expert, bounded by expert capacity*

Remark 4.9 (Trade-off with stochastic MoE). In standard Mixture-of-Experts with learned gating, routing quality depends on the learned router and its auxiliary losses. Token-Routed MLP removes router learning and load-balancing losses, but it also gives up content-adaptive expert selection. The empirical question is whether lexical partitions plus a shared expert provide enough conditional specialization to offset this restriction.

5 Implementation

5.1 Technical Specifications

Our implementation uses PyTorch 2.0+ with the following optimizations:

Table 2: Implementation choices

Component	Technical Choice
Attention	SDPA / Flash Attention
Positional Encoding	RoPE ($\theta = 10000$)
Normalization	RMSNorm + QK-Norm
Activation	SiLU (SwiGLU)
Precision	BF16 (training and inference)

5.2 1.5B Model Configuration

Table 3: COMPLEXITY-DEEP 1.5B model configuration

Parameter	Value
Layers (L)	24
Dimension (d_{model})	2048
Attention heads (n_h)	16
KV heads (n_{kv})	4 (GQA ratio 4:1)
Intermediate dimension	8192
Experts ($n_{experts}$)	4
Vocabulary	32000
Max context	4096
Total parameters	**1.5B**

Table 4: COMPLEXITY-DEEP 300M model configurations (iso-params comparison)

Parameter	Token-Routed (MoE)	Dense Baseline
Layers (L)	18	18
Dimension (d_{model})	1024	1024
Attention heads (n_h)	16	16
KV heads (n_{kv})	4 (GQA ratio 4:1)	4 (GQA ratio 4:1)
Routed intermediate dimension	256 total	—
Expert intermediate	64	—
Shared expert intermediate	3840	—
Experts ($n_{experts}$)	4, top-$k = 2$	1 (dense)
Shared/routed gates	1.0 / 0.1	—
Vocabulary	32000	32000
Max context	2048	2048
Total parameters	**306.5M**	**306.5M**
Active MLP path/token	shared + 2 routed experts	dense SwiGLU

5.3 300M Model Configuration

For the corrected iso-parameter scaling comparison (Section 7.4), we train two architectures at ∼300M parameters (Table 4): a residual Token-Routed model (306.5M, 4 experts, top-$k = 2$, large shared expert and small routed branch) and a dense SwiGLU baseline (306.5M, $d_{ff} = 4096$). Both share the same backbone (18 layers, $d_{model} = 1024$, GQA 16/4, RMSNorm, QK-Norm), tokenizer, sequence length, optimizer, warmup schedule, and FineWeb-Edu token budget.

6 Experiments

6.1 Pilot Study

An exploratory 1.5B-parameter model trained on 7B tokens motivated the architectural simplification: the adaptive residual scaler was removed (redundant with Adam), Zipf-balanced routing was added, and the Shared Lexical Expert was introduced. The evidence in this paper is therefore centered on the corrected 300M iso-parameter comparison (Table 4).

6.2 Training Recipe

Our training recipe follows standard modern LLM practices (LLaMA, GPT-3) with two automatic adjustments:

Dynamic warmup. Instead of a fixed number of warmup steps (which can represent 52% of training for short runs), warmup is automatically set to **5% of total steps**. This ensures a constant warmup/training ratio regardless of run duration.

GPT-style initialization. Residual output projections (`o_proj`, `down_proj`) are initialized with reduced standard deviation:

$$\sigma_{\text{residual}} = \frac{0.02}{\sqrt{2 \times L}} \tag{21}$$

where L is the number of layers. This prevents the residual stream from growing with depth (Radford et al., 2019).

Other hyperparameters. Weight decay = 0.1 (standard GPT-3/LLaMA), AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping = 1.0, BF16 precision, cosine scheduler with min_lr = 10% of peak.

6.3 Zipf-Balanced Routing

The original modulo routing ($\text{expert}(t) = t \bmod n$) distributes the *vocabulary* uniformly but not the *corpus*. Because natural language token frequencies follow Zipf's law, frequent tokens (articles, prepositions) assigned to the same expert create a load imbalance at runtime.

Zipf-balanced greedy bin-packing: We sort the vocabulary by empirical frequency (computed from a sample of the training corpus) and assign each token to the expert with the lowest current total load:

$$\text{expert}(t) = \arg\min_{e} \sum_{t' \in T_e} \text{freq}(t'), \quad \text{for } t \text{ in descending frequency order} \tag{22}$$

where T_e is the set of tokens already assigned to expert e. Unlike round-robin (which can leave large Zipf-induced imbalances), greedy bin-packing balances the expected load under the frequency estimate used to construct the table. The mapping remains deterministic and is computed once before training; realised per-batch and final-run utilization can deviate from exactly 25% because the training stream is finite and non-stationary.

7 Discussion

7.1 Comparison with Existing Architectures

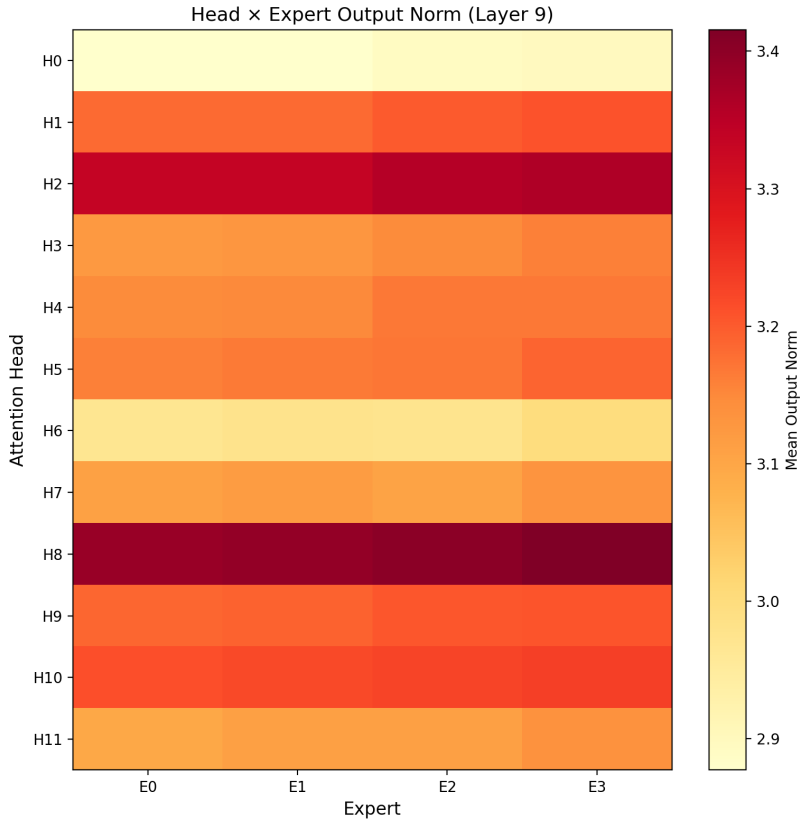

Figure 3: Attention head × expert heatmap (median layer). Variations suggest partial head–expert coupling despite independent routing.

Table 5: Architectural comparison

Architecture	**Routing**	**Load balancing loss**
Transformer	No	N/A
Switch Transformer	Soft	Required
Mixtral	Top-2	Required
COMPLEXITY-DEEP	**Hard lexical**	**Not used**

7.2 Token-Routing Analysis

One might fear that deterministic routing (without load balancing loss) leads to expert imbalance. Our analysis indicates that Zipf-balanced routing keeps all experts active:

- **Near-balanced utilization**: In the 300M run, final observed expert utilization is 0.2481/0.2635/0.2481/0.2403, with no dead experts.
- **Balanced norms**: Expert norms remain close across routed branches.
- **No collapse**: All 4 experts remain differentiated (inter-expert difference $\sim$0.022)

Functional specialization of experts: We measure specialization via *per-expert perplexity* rather than cosine similarity (which is a geometric artifact expected in high dimensions). Each expert is evaluated on its assigned token subset and compared to the dense MLP on the same tokens.

Table 6: Token-Routed MLP vs soft-routing MoE

Criterion	**Token-Routed**	**Standard MoE**	**Advantage**
Expert balance	Frequency bin-packing	Aux. loss required	Token-Routed
Specialization	Functional (PPL/expert)	Learned (softmax)	Depends
Deployment	No gating network	Gating + dispatch	Token-Routed
Determinism	100%	No (softmax)	Token-Routed
Shared Expert	Yes (common patterns)	Optional	Token-Routed
Specialization type	Lexical	Semantic	Standard MoE

7.3 Controlled 100M Ablation Preview (1B tokens)

We also ran a seven-way 100M ablation suite designed to isolate the contribution of the shared dense path and the lexical routed residual branch. Each run uses the same 2×B200 token budget: 954 steps × 2 GPUs × batch/GPU 256 × sequence length 2048 = 1.0003B tokens. Table 7 reports all seven runs: dense residual, Zipf shared, Zipf no-shared, modulo shared, random shared, round-robin shared, and shared-only. These results are an ablation-scale preview, not a replacement for the 300M scaling comparison below.

The ordering is informative. Zipf shared is the best variant and is ahead of the matched dense residual baseline in both logged final training loss and validation loss: at the last logged training point (step 950, 0.996B tokens), Zipf shared reaches 4.8205 versus 4.8244 for dense; at the best validation checkpoint (step 750, 0.786B tokens), Zipf shared reaches 4.8887 versus 4.8958 for dense. Modulo and round-robin shared are close but remain behind dense, random shared is substantially worse, shared-only underperforms dense despite high throughput, and Zipf no-shared degrades most strongly. This pattern suggests that lexical token identity is a meaningful routing object only when paired with a shared dense backbone and a frequency-aware partition: the routed branch acts as a residual lexical specialization path, not as a replacement for dense MLP computation or as a generic capacity add-on.

Dense is faster than Zipf shared in the current training implementation (397k vs. 321k tok/s near the end), while shared-only is fastest (402k tok/s) but much worse in loss. These are therefore matched-token quality comparisons rather than matched-wall-clock efficiency claims.

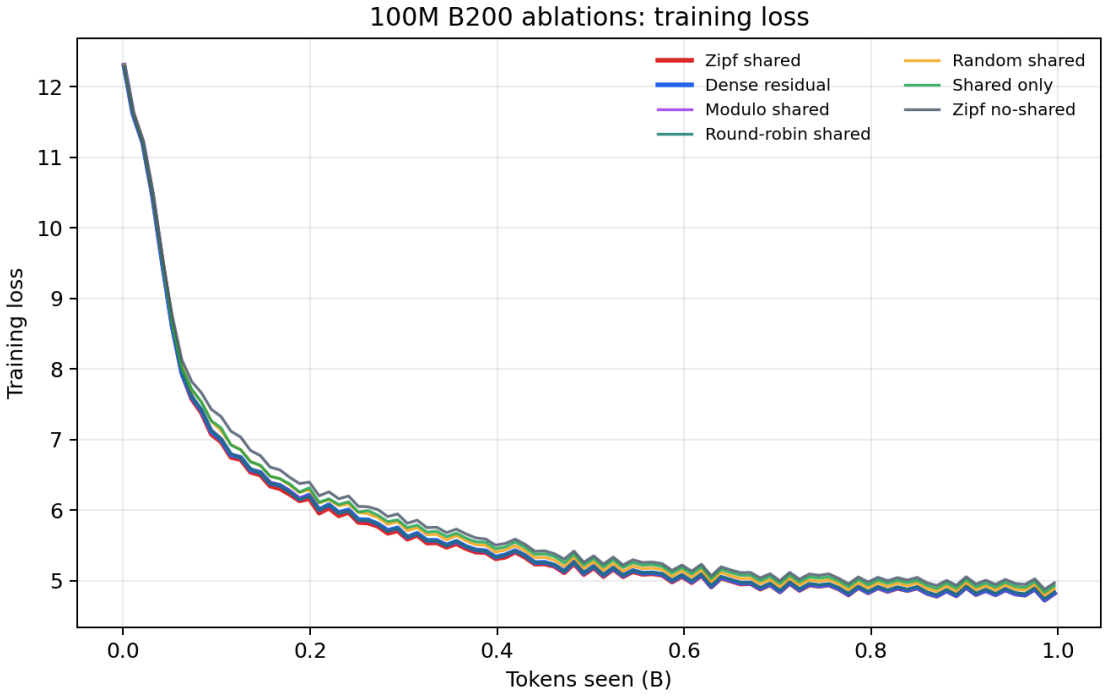

Figure 4: 100M ablation training curves on 2×B200, matched at 1.0003B tokens. Zipf shared finishes slightly ahead of the matched dense residual baseline; modulo and round-robin shared remain close but behind, while random shared, shared-only, and Zipf no-shared underperform.

Table 7: 100M ablation runs on 2×B200. All rows use the same 1.0003B-token budget; best validation loss occurs at step 750 for all runs.

Variant	**Train loss @ 950**	**Best val. loss @ 750**	**Tok/s near end**
Zipf shared	**4.8205**	**4.8887**	321k
Dense residual	4.8244	4.8958	397k
Modulo shared	4.8320	4.9016	321k
Round-robin shared	4.8384	4.9072	321k
Random shared	4.8875	4.9577	318k
Shared-only	4.9285	5.0012	**402k**
Zipf no-shared	4.9693	5.0393	334k

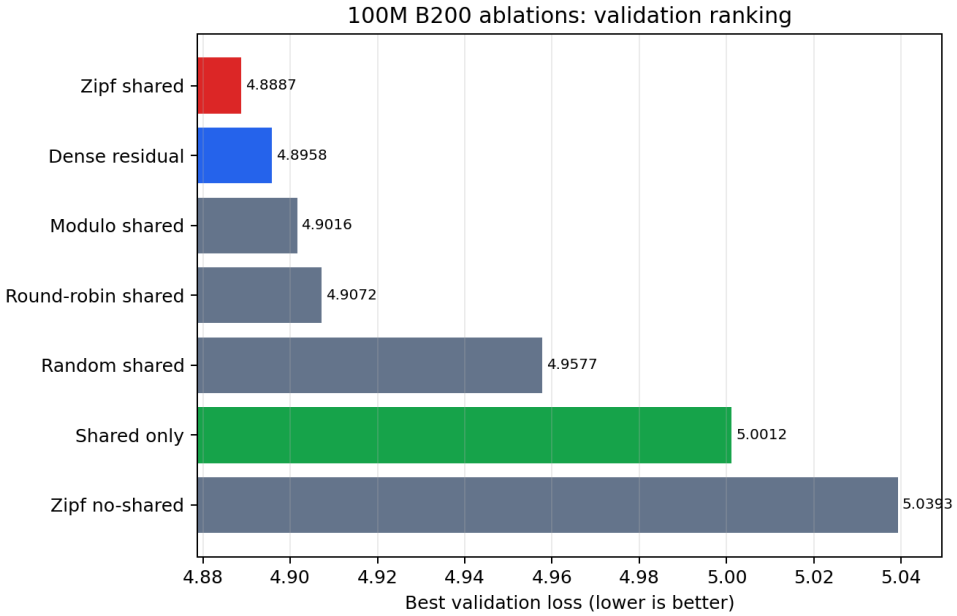

Figure 5: Best validation loss ranking for the 100M B200 ablations. Lower is better. Zipf-balanced shared routing is the best variant and the only routed variant ahead of the dense residual baseline; shared-only does not explain the gain.

7.4 Scaling Comparison (300M, 8B tokens)

To validate the corrected architecture at larger scale, we train iso-parameter models (Table 4) on an 8B-token FineWeb-Edu budget. Both runs use the same global batch of 1,048,576 tokens/step (8 GPUs $\times$ batch 64 $\times$ sequence length 2048), so matched raw step number equals matched tokens seen, and no token-grid interpolation is required.

The two runs start from comparable random-init loss (TR 10.58, dense 10.54). The Token-Routed model evaluated here is the shared-plus-routed residual variant: most MLP capacity is in the shared dense path, while a smaller Zipf-routed branch supplies lexical specialization. It trails the dense baseline during the early phase while its experts are still random and not yet specialized: at step 100 ($\approx$105M tokens) the gap is $+0.177$ in favor of dense, peaking near $+0.31$ around step 40. The gap shrinks overall as Zipf-routed experts specialize: the first logged train-loss win appears at step 740 ($\approx$776M tokens), and the first matched validation win appears at step 750. The lead peaks near -0.023 around step 2000, then narrows slightly as both runs approach convergence: at step 7620 ($\approx$7.99B tokens, end of budget) Token-Routed reaches train loss **2.9231** versus **2.9324** for dense, and the smoothed final gap over the last 50 logged steps is -0.0163 in favor of Token-Routed. Expert utilization remains tightly balanced throughout (0.2481/0.2635/0.2481/0.2403, zero dead experts), confirming that the win is not driven by expert collapse or routing imbalance.

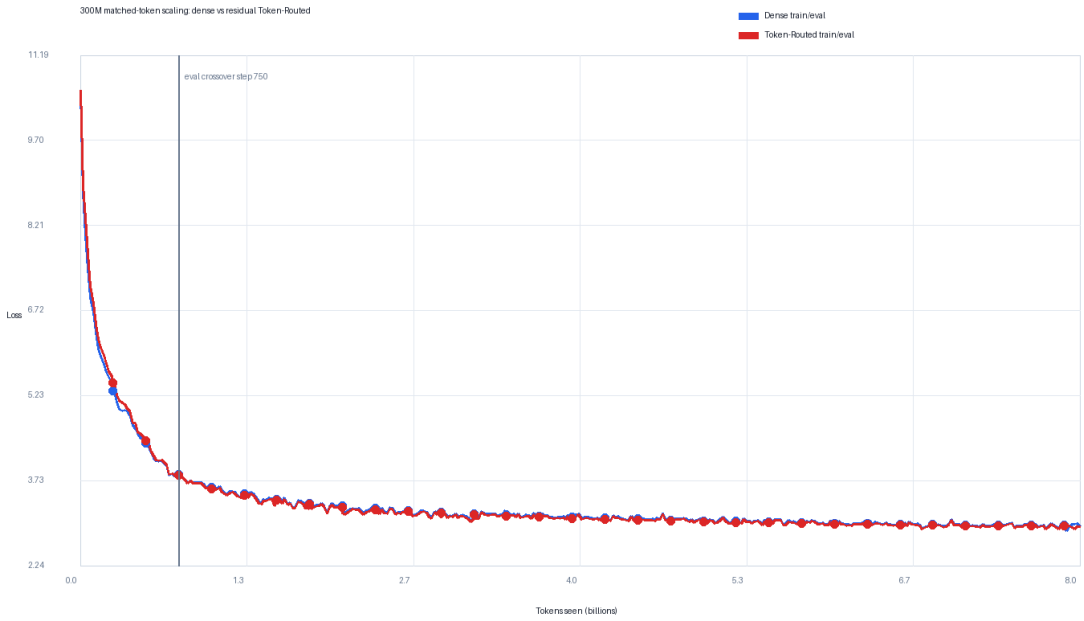

Figure 6: Corrected 300M scaling curves at iso-batch (1,048,576 tokens/step). Token-Routed (red) trails dense (blue) during the first ≈700 steps while experts specialize, reaches parity, then crosses over: by step 1000 (≈1.05B tokens) Token-Routed leads.

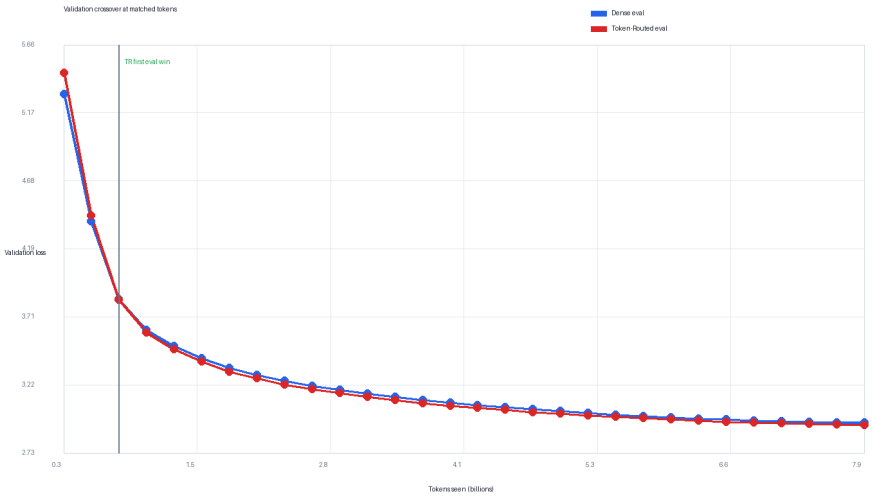

Figure 7: Matched validation-loss comparison. Token-Routed pays an early specialization cost at the first two validation points, then crosses the dense baseline at step 750 (≈786M tokens) and remains ahead through the 8B-token budget.

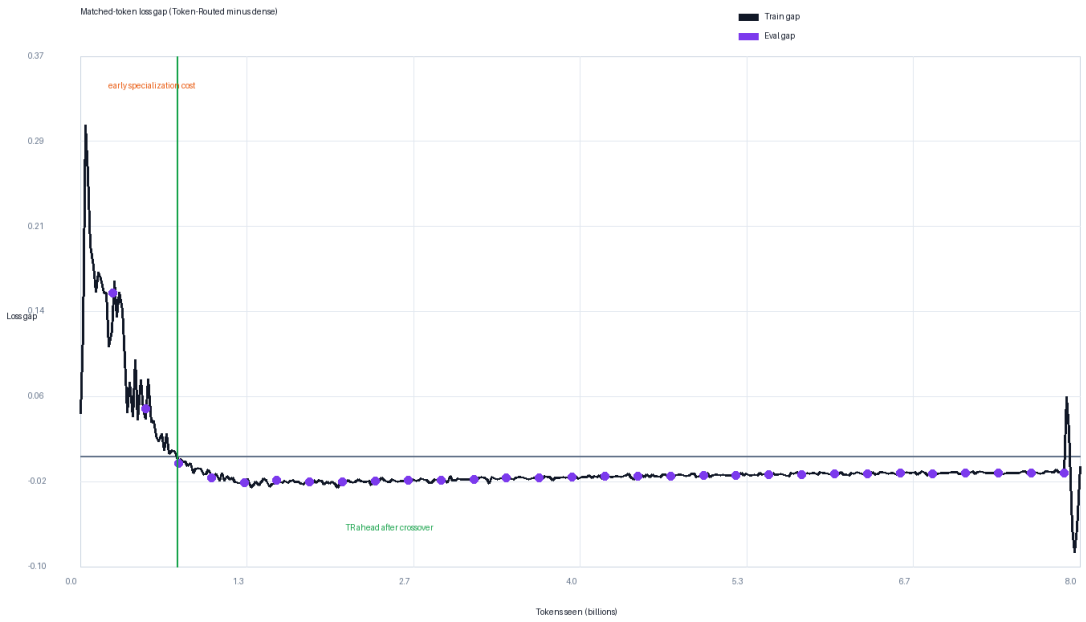

Figure 8: Training-loss gap at matched step (= matched tokens, iso-batch), computed as Token-Routed loss minus dense loss. Negative values indicate Token-Routed ahead. The gap is positive (TR behind) during the early specialization phase and first turns negative at the logged step 740.

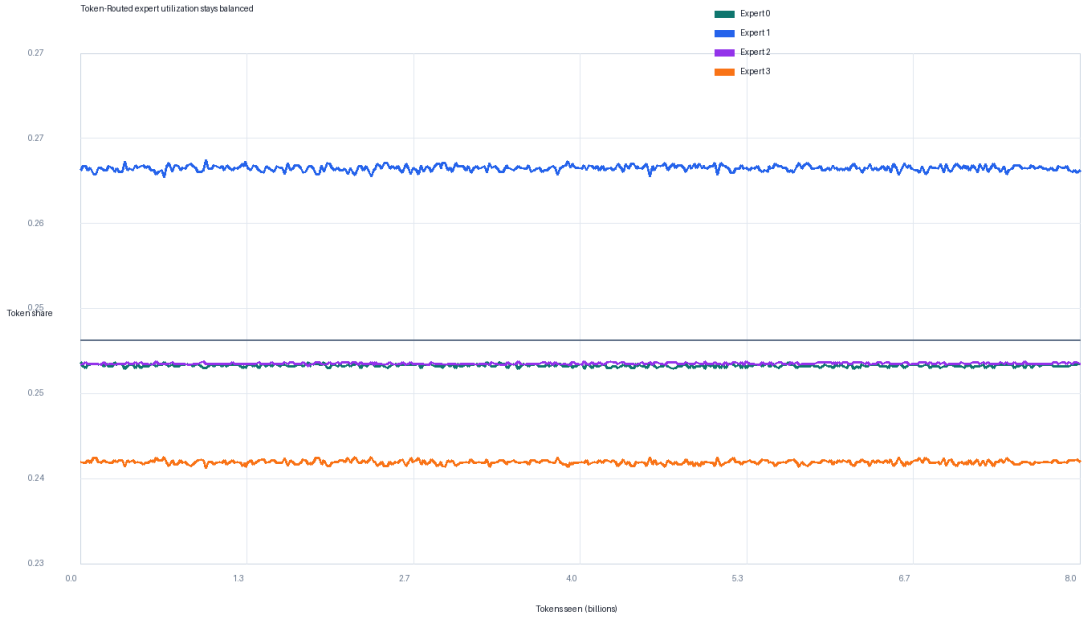

Figure 9: Expert utilization during the corrected 300M Token-Routed run. All four experts remain active and near-balanced; the observed matched-budget win is not caused by expert collapse.

Table 8: 300M scaling comparison at iso-batch (1,048,576 tokens/step), full 8B-token run. Train loss is reported at matched step. Negative gap indicates Token-Routed ahead. Last row is the smoothed mean of the final 50 logged steps.

Step	**Tokens seen**	**Dense (306.5M)**	**TR (306.5M, k=2)**	**Gap (TR − Dense)**
100	105M	6.6698	6.8464	+0.177
500	524M	4.4450	4.4796	+0.035
700	734M	3.8567	3.8619	+0.005
1000	1.05B	3.5500	**3.5324**	**−0.018**
2000	2.10B	3.1953	**3.1720**	**−0.023**
4000	4.19B	3.0925	**3.0738**	**−0.019**
6000	6.29B	3.0061	**2.9906**	**−0.015**
7620	7.99B	2.9324	**2.9231**	**−0.009**
last 50 (smoothed)	—	2.9446	**2.9283**	**−0.016**

Training throughput. The 300M Token-Routed model uses a large shared branch plus a small top-$k = 2$ routed branch, so it is not intended as a pure top-1 speed benchmark. Both runs were trained on 8×B300, dense reaching approximately 0.95M tokens/s and Token-Routed approximately 0.75M tokens/s at identical global batch (1,048,576 tokens/step). All quality comparisons above are at matched tokens seen. A matched-wall-clock comparison would answer a different question—whether the current implementation is more compute-efficient—and would favor the faster dense baseline unless the routed residual branch is further optimized. We therefore report throughput separately and do not claim wall-clock efficiency. The routing table lookup itself is $O(1)$ and does not require learned-router synchronization.

7.5 Inference Sanity Check

To check that deterministic lexical routing is compatible with standard serving stacks, we also benchmarked an earlier small Token-Routed checkpoint on vLLM 0.18 with PagedAttention and CUDA graphs. This serving result is not used as evidence for the corrected 300M scaling comparison above, and it should not be interpreted as a compute-efficiency claim for the 300M checkpoint. It only verifies that deterministic per-token routing can be deployed without a learned router or all-to-all dispatch.

On a single NVIDIA RTX PRO 6000 GPU (96 GB), serving 100 concurrent requests at 10 RPS with 128 input tokens and 1,900 output tokens, the checkpoint reaches a sustained throughput of **8,078 tokens/s** (peak 10,179 tokens/s), with median time-to-first-token (TTFT) of 29.3 ms and median inter-token latency (ITL) of 7.9 ms (Figure 10). Updated inference measurements for the corrected 300M checkpoint remain future work.

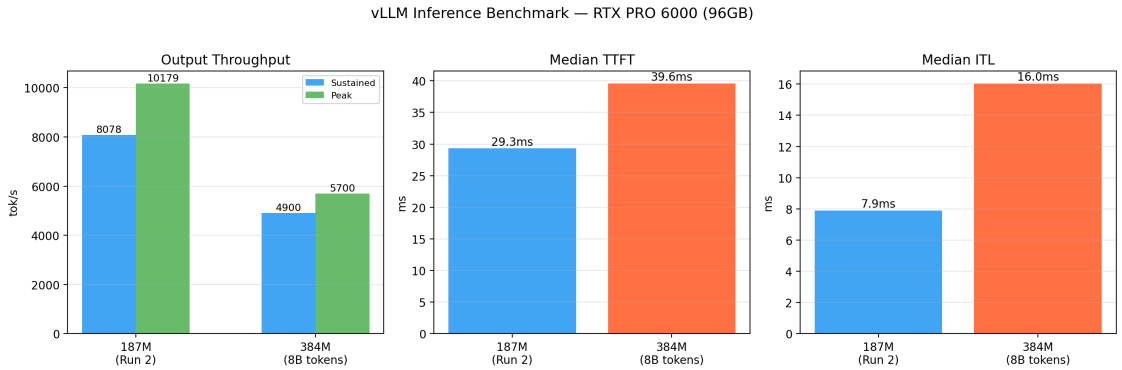

Figure 10: Inference sanity check on a single NVIDIA RTX PRO 6000 (96 GB). 100 requests at 10 RPS, 128 input tokens, 1,900 output tokens. Sustained throughput: 8,078 tok/s; peak: 10,179 tok/s; median TTFT: 29.3 ms; median ITL: 7.9 ms. This benchmark uses an earlier small checkpoint and is reported only as a deployment compatibility check.

7.6 Limitations and Future Work

- **Single-seed scaling result**: The corrected 300M run is a full 8B-token matched-budget comparison, but it is still a single seed per architecture. Additional seeds would quantify whether the final -0.0163 smoothed gap is robust to initialization variance.

- **Distribution-specific routing table**: Zipf bin-packing uses an empirical frequency estimate from the training distribution. Domain shifts can change realised expert utilization and should be measured when transferring the routing table.

- **Standardized benchmarks**: Zero-shot evaluation on ARC-Easy, HellaSwag, and MMLU via `lm-evaluation-harness` should be rerun on the corrected 300M checkpoints.

8 Conclusion

We presented COMPLEXITY-DEEP, a language model architecture centered on Token-Routed MLPs with Zipf-balanced greedy bin-packing and a Shared Lexical Expert for deterministic assignment without auxiliary load-balancing losses.

In the corrected iso-parameter 300M scaling run at iso-batch (1,048,576 tokens/step, full 8B-token budget), the residual Token-Routed variant pays a transient specialization cost during the first $\approx$700 steps (peak gap $+0.31$ near step 40), first wins on logged train loss at step 740 and on validation loss at step 750, then stays ahead until the end: the smoothed final train-loss gap over the last 50 logged steps is -0.0163 in favor of Token-Routed (2.9283 vs 2.9446). These results support deterministic lexical routing as a promising matched-token quality signal when used as a residual lexical branch on top of a shared dense MLP backbone. They should not be read as evidence that routing alone replaces dense MLP computation. Multi-seed scaling, compute-normalized comparisons, and updated inference measurements for the corrected 300M checkpoint remain important next steps.

Broader Impact Statement

This work introduces architectural innovations for language models. While these models have broad applications, they also carry risks of misuse. We release the model weights to enable research while encouraging responsible use.

Reproducibility Statement

To facilitate reproducibility and review, we provide the complete model architecture implementation as supplementary material (`supplementary_code.zip`). The code (PyTorch 2.0+) will be made publicly available upon acceptance.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512, 2023.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2018.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

A Training Curves

Additional training curves and analysis figures are available in the supplementary material.